ENGENHARIA DE RAG CORPORATIVO

Arquitetura de Recuperação Vetorial, Grounding Anti-Alucinação, FinOps e APIs de Produção

Stack Tecnológica: Google GenAI SDK (Gemini), ChromaDB, FastAPI, Pydantic, PyPDF

Objetivo: Capacitar desenvolvedores e engenheiros de IA a construir, orquestrar e monitorar pipelines de RAG corporativos resilientes a falhas e controlados financeiramente.

1. O Conceito Fundamental de RAG

Retrieval-Augmented Generation (RAG) é um padrão arquitetural projetado para resolver a principal fraqueza dos Modelos de Linguagem (LLMs): a falta de conhecimento sobre dados privados e o risco de *alucinação* (respostas inventadas com tom assertivo).

Em vez de tentar retreinar ou fazer *Fine-Tuning* do modelo com dados da empresa — o que é financeiramente proibitivo, lento e desatualizado no minuto seguinte —, o RAG atua como um exame com consulta: quando o usuário faz uma pergunta, recuperamos do banco apenas os parágrafos relevantes e os injetamos diretamente no prompt da LLM.

Critério	Fine-Tuning Tradicional	Arquitetura RAG
Custo de Manutenção	Altíssimo (re-treinamento constante com GPUs)	Baixo (apenas ingestão vetorial incremental)
Atualização dos Dados	Lenta (depende de ciclos de treino)	Instantânea (basta indexar no ChromaDB)
Auditoria de Fontes	Impossível (modelo é uma caixa-preta)	Total (o sistema retorna a fonte exata consultada)

2. Ingestão de Dados: Extração e Chunking Estratégico

Modelos de embedding e LLMs têm janelas finitas e perdem acurácia quando submetidos a documentos gigantescos de uma só vez. A estratégia de **Chunking com Janela Deslizante (Sliding Window)** garante que o texto seja quebrado em blocos semanticamente assimiláveis.

O Papel Vital do Overlap (Sobreposição): Sem overlap, uma frase importante como *'A bonificação será de R$ 350,00'* pode ser cortada exatamente entre o valor e a condição, quebrando a integridade semântica da resposta.

```
def fazer_chunking(texto: str, chunk_size: int = 1000, overlap: int = 100):
chunks = []
inicio = 0
while inicio < len(texto):
fim = inicio + chunk_size
chunks.append(texto[inicio:fim])
inicio += (chunk_size - overlap) # Retrocesso de segurança
return chunks
```

3. Embeddings, Distância de Cosseno e Resiliência (429)

Um modelo de embedding converte frases em coordenadas em um espaço multidimensional. Textos com sentidos semelhantes apontam na mesma direção vetorial. A métrica utilizada é a **Distância de Cosseno**: quanto mais próximo de 0, maior o parentesco semântico.

Engenharia de Resiliência: APIs em nuvem possuem cotas de RPM (Requisições por Minuto). Para evitar falhas em lote, nosso gerador implementa *Exponential Backoff* (espera crescente caso receba HTTP 429) e pausas entre batches.

Cálculo da Similaridade por Cosseno:
*sim(u, v) = (u · v) / (||u|| * ||v||)*
• Valor próximo de 1.0: Sentidos praticamente idênticos.
• Valor próximo de 0.0: Textos ortogonais sem relação.

4. Armazenamento Vetorial e Prompt Grounding (Anti-Alucinação)

O ChromaDB é configurado com metadata={'hnsw:space': 'cosine'}. A persistência local garante que você não precise re-vetorizar seus documentos a cada inicialização da aplicação.

A Técnica de Grounding Estrito: Para forçar o Gemini a nunca alucinar, amarramos a instrução do sistema ao contexto extraído. Se a resposta não constar estritamente nos trechos recuperados, a LLM é instruída a declinar a resposta.

```
# Prompt Ancorado (Grounding Rígido)
prompt = f"""Você é um assistente corporativo.
Responda à pergunta do usuário utilizando ESTRITAMENTE as informações do contexto abaixo.
Se a resposta não estiver no contexto, diga que a informação não consta nos documentos.

CONTEXTO RECUPERADO:
{contexto}

PERGUNTA:
{pergunta}

RESPOSTA:"""
```

5. Observabilidade e FinOps: Monitoramento de Custo Real

Em produção, colocar uma LLM sem controle de custos é garantia de prejuízo orçamentário. O nosso motor RAG extrai a telemetria do usage_metadata e divide as latências por fase.

Fase Medida	Mecanismo de Coleta	Objetivo / Métrica
Embeddings	time.perf_counter()	Gargalo de I/O de rede com a Google
Busca ChromaDB	time.perf_counter()	Eficiência do índice vetorial HNSW local
Inferência LLM	usage_metadata	Contagem exata de tokens de Input e Output
Custo em Dólares	Fórmula FinOps de precificação	Cálculo: (Prompt/1M * $0.075) + (Resp/1M * $0.30)

6. Exposição via API REST (FastAPI, Pydantic e Concorrência)

Para integrar o RAG a portais corporativos, utilizamos o FastAPI com validação estrita via Pydantic.

O Ponto Crítico de Concorrência (Sync vs Async): Funções que realizam chamadas síncronas pesadas ou que utilizam time.sleep (como no retry da IA) **NÃO** devem ser declaradas como async def, sob risco de congelar o Event Loop do servidor. No FastAPI, declará-las com def tradicional faz o framework alocar automaticamente uma threadpool dedicada, mantendo o servidor 100% responsivo.

Método	Rota	Payload / Formato	Ação Executada
GET	/health	Vazio	Health check de conectividade
POST	/ingest	JSON (IngestaoRequest)	Segmenta e indexa texto puro
POST	/ingest/arquivo	Multipart Form (.PDF / .TXT)	Extrai bytes, valida e indexa arquivo
POST	/query	JSON (PerguntaRequest)	Busca, gera resposta e emite métricas